2025

Smart Scheduler for Carleton Engineering Students

FALL PROGRESS REPORT



Smart Scheduler

Report Citation

Smart Scheduler for Carleton Engineering Students.

Smart Scheduler.

December 05, 2025.

G67 - Proff. Jerome Talim Team

Members:

Anas Ayoubi (101226450)

Hakeem Khan (101233039)

Pietro Adamvoski (101238885)

Safiullah Rattar (101245709)

TABLE OF CONTENTS

1.0 Introduction 3
2.0 Project Goals From the Proposal 3
3.0 Work Completed During the Fall Term 4
3.1 Backend Structure and System Setup 4
3.2 Database Design and Course Information 5
3.3 Scheduling Engine 5
3.4 Prerequisite Checking System 6
3.5 Academic Audit Parsing 6
3.6 Frontend Pages and User Flow 6
4.0 How Our Progress Matches the Proposal 7
5.0 Changes From the Original Plan 8
6.0 Winter 2026 Plan 8
7.0 Challenges and Lessons Learned 9
8.0 Team Workflow and Collaboration 10
9.0 Future Improvements and Long-Term Vision 10
10.0 Testing Conducted So Far 12
11.0 Conclusion 13

1.0 INTRODUCTION

During the Fall 2025 term, our group has been working on the Smart Scheduler project, a web-based tool designed to help engineering students plan their courses more easily. The main idea behind our system is to take the confusing and time-consuming parts of course selection checking prerequisites, comparing all the lecture and lab options, figuring out which sections overlap, and trying to build a clean schedule. Our goal is to automate as much of that work as possible. Students often spend hours trying to piece things together manually, and our goal is to make that entire process much simpler, faster, and much less stressful.

This progress report explains what we have completed so far, how our current work compares to what we originally proposed, what changes or improvements happened along the way, and what we plan to continue working on during the Winter term. Although we are not finished the full system yet, the major building blocks are already working, and many of the harder technical challenges have been solved, which gives us confidence moving into the second half of the project.

2.0 Overview of the Project Goals

In our original proposal, the goal was to build a system that lets students upload their academic audit, check course eligibility, choose the courses they want to take next term, and generate a list of schedules with no conflicts. The proposal also described extra features like a ranking system, elective recommendations, and an option to export schedules.

For the Fall term, the main plan was to focus on the core systems behind the scenes: the backend, the database structure, the scheduling engine, and the prerequisite checking logic. We also wanted to get the basic frontend pages working so we could connect everything in the Winter term.

These goals helped us stay focused and made it easier to track our progress throughout the semester.

3.0 Progress Made During the Fall Term

3.1 Backend Structure and System Setup

One of the first tasks we completed this term was setting up the backend. This included creating routes for schedule generation, course searching, and audit parsing. The backend is now able to talk to the database, run the scheduling algorithm, and send results to the frontend.

While the proposal originally assumed that we might use one unified framework, we decided to separate the frontend and backend into two parts. This made teamwork easier because different members could work on different areas without causing conflicts. It also made debugging simpler. This didn't change any of the system's goals, it just made development smoother for us. The backend is stable, and most of its structure is ready for the remaining Winter work.

3.2 Database Design and Course Information

The next part of the project involved setting up the database. The database holds all course information, including the full course list, lecture and lab sections, instructors, times, and

prerequisite rules. We loaded most core engineering courses into the system and tested them against real timetable data.

We also had to carefully design how prerequisites are stored because some courses have simple requirements while others allow multiple pathways or have year-standing rules. This took some extra effort, but the final design works well and gives us the flexibility to support a wide range of courses.

The database is now one of the strongest parts of the project and supports all the features we built so far.

3.3 Scheduling Engine

The scheduling engine is one of the most important parts of the project, and we completed a working version this term. The engine looks at the selected courses and explores every possible combination of lecture, lab, and tutorial sections to build schedules that have no overlapping times.

We used a backtracking approach with pruning to avoid checking impossible combinations. This makes the system fast and helps it generate schedules in just a few seconds. Since students usually take four or five courses in a term, this performance is more than enough for the final version of the tool.

Another achievement was getting the engine to correctly group multi-component courses. Instead of randomly mixing different lectures and labs, it only uses valid sets that belong together. This makes the generated schedules accurate and realistic.

3.4 Prerequisite Checking System

We finished building the prerequisite checker this term. This system reads the rules each course requires and checks whether the student meets them. It supports common cases like needing one specific course, needing one of multiple courses, requiring a certain grade, or needing to be in a specific year level.

When a student selects a course, the system can tell them right away whether they meet the requirements. This also prevents the scheduler from creating schedules with courses the student wouldn't be allowed to register for. Getting this working properly took time, especially because course rules are not always written the same way, but the system now handles the majority of them accurately.

3.5 Academic Audit Parsing

The academic audit parser was another important part of the project. Students can upload the HTML version of their audit, and the system extracts their name, student ID, completed courses, and courses in progress. The HTML parsing works well and gives us exactly what we need.

We tested PDF parsing as well, but because PDF documents can be formatted in different layouts, it is less reliable. For now, HTML audit parsing is the main supported format, and PDF parsing may become an optional improvement if time allows.

3.6 Frontend Pages and User Flow

We made strong progress on the frontend this term. There are three main pages that students will use, and all of them are functioning.

The first page lets students upload their academic audit or manually enter completed courses. After uploading, the system shows what it extracted so the student can confirm everything looks right.

The second page lets students search for and choose the courses they want to take next term. The page also displays prerequisite information so students immediately know whether they are eligible for each course.

The final page shows the schedule results in a weekly timetable. Students can scroll through the different schedule options, check details like instructors and section numbers, and compare schedules visually.

We also added helpful features like time-range filters (earliest and latest class times), support for different scheduling modes, and future space for an “AI-recommended” label. These were not in the original proposal but make the tool much more practical for real students.

4.0 How Our Progress Matches the Proposal

Almost everything we originally planned for the Fall has been completed. The scheduling engine, prerequisite checker, backend system, and main frontend flow are all functional. A few features, like ranking schedules and recommending electives, are planned for the Winter because they build on top of the systems we created this term.

There were small changes in our approach, like splitting the backend and frontend, but these did not change any goals. They simply made development easier and gave us a cleaner structure to work with.

5.0 Changes From the Original Plan

There were a few adjustments and additions this term. Splitting the frontend and backend helped us work more efficiently, even though this wasn't mentioned in the proposal. We also added several features we felt would make the system better for real students, such as time-preference filters and multiple scheduling modes.

Some tasks, like ranking schedules based on preferences, are moved to the Winter term. This isn't because we fell behind, but because we wanted to focus on getting the core features completely stable before adding enhancements.

6.0 Winter 2026 Plan

During the Winter term, we plan to finish the ranking system so the scheduler can sort schedules based on preferences like avoiding early mornings or building tighter days. We will also finish the elective recommendation system, which will help students understand which optional courses they can choose that still fit their program requirements.

Another big part of Winter will be testing the system with real users, improving the UI, and making everything feel smooth. We will also add export options so students can download their schedules or get a list of CRNs to use during registration.

By the end of the Winter term, our goal is to have a clean, polished version of the system that we can confidently present as our final capstone project.

7.0 Challenges and Lessons Learned

Throughout the Fall term, we ran into a few challenges that slowed us down at different points, but they also helped us understand our system better and make smarter decisions for the Winter term. One of the bigger challenges was figuring out how to store prerequisite rules in a way that made sense. At first, we tried to represent every rule as simple text, but that didn't work well when we ran into courses that have several different "paths" or year-standing requirements. We ended up redesigning our approach so the system could break these rules into smaller pieces and check them one by one. This took time, but the result is much more reliable.

Another challenge came from the scheduling engine. When we first built it, we noticed the algorithm was checking way too many combinations and sometimes took a long time to finish. We had to spend time improving how the engine "cuts off" combinations that obviously won't work. Once we added pruning, the engine became much faster and more consistent. This kind of showed us that even though algorithms look good on paper, they usually need adjustments when you actually run them. Working with academic audits was also a learning experience. While the HTML files were easy to parse, PDF files had many different layouts and sometimes lacked consistent structure. This taught us that supporting multiple formats is harder than expected, and it also made us appreciate why many applications stick with a single preferred format. On the frontend side, a challenge was making the user flow feel natural. It can be easy to build pages that work, but harder to build pages that feel intuitive. We tested multiple layouts before choosing the current one. These kinds of challenges taught us that development is not only about code working, but also about the user having a smooth experience.

Overall, these challenges helped us grow as a team. We communicated better, divided tasks more clearly, and learned how to solve problems one step at a time. Everything we learned this term sets us up well for the Winter portion of the project.

8.0 Team Workflow and Collaboration

One of the things we focused on this term was improving how we worked together as a team. Since this project has many different parts, we had to find a way to divide tasks without blocking one another. Early in the term, we noticed that having everything inside one project made merging changes complicated, so we switched to separating the backend and frontend. After doing this, collaboration became easier because each of us could focus on the parts we were more comfortable with.

We held weekly meetings to check on each other's progress and discuss any issues. These meetings helped us stay aligned and avoid misunderstandings about what needed to be done next. Whenever someone ran into a problem—whether it was a bug, a confusing prerequisite rule, or database structure questions—we would talk it through as a group until we found a solution.

We also learned the importance of documenting our decisions. If someone was working on the scheduler, for example, they would write down how it worked so other members could understand it later. This helped reduce dependency on any one person and kept development moving even when someone was busy or unavailable.

Overall, our teamwork has improved a lot since the proposal stage. We communicate better, share tasks more evenly, and understand the strengths each member brings to the project.

9.0 Future Improvements and Long-Term Vision

Even though our goal is to complete the final version by April, there are several features we think could be added in the future if the project were to continue after Capstone. One idea is to integrate real-time enrollment data so students can see whether a section is full or almost full. Another

improvement would be connecting the system directly to Carleton Central so students could import their schedule instead of manually copying CRNs. We also think it would be helpful to support multiple years of study, not just first-year engineering students. This would require loading more data and handling more complex graduation requirements, but it would make the system useful for a much larger group of students. Another possibility is to improve the ranking system by using machine learning to learn what types of schedules students prefer based on patterns. While this is beyond the scope of the current project, it could turn the Smart Scheduler into a more personalized tool in the future. These ideas aren't required for our Capstone project, but they show that the system has room to grow later on if someone wanted to keep building it

10.0 Testing Conducted So Far

Although most of the full testing will happen in the Winter term, we have already done basic testing on the main features. For example, we tried loading different academic audits to make sure the parser reads all the courses correctly. We also tested the scheduling engine with various course combinations, including difficult ones with lots of overlapping sections, to make sure it produces valid schedules consistently.

We checked the prerequisite system with several edge cases, such as courses with multiple requirement paths or those needing minimum grades. These early tests helped us catch small issues before they became bigger problems. Overall, the early testing gave us confidence that our core systems are stable enough to support the more advanced features coming next term.

11.0 Conclusion

The Fall term went very well, and we were able to complete the major parts of the Smart Scheduler project. The backend, database, scheduling engine, prerequisite checker, and main frontend pages are all up and running. These were the hardest and most important parts of the project, and getting them done puts us in a strong position for the Winter term. We still have features to add and improvements to make, but the foundation is solid. Our team feels confident about finishing the rest of the project on time and delivering a finished system by April.

Last Page of Document